

Interwise — Seminar Presentation Guide

Oct 7, 2026 · @Sobhan

Project overview

Interwise is an AI mock-interview platform. It writes 15 interview questions tailored to your resume or role. You answer each one by typing or speaking, and every answer gets a score from 1 to 100 with written feedback.

- **Problem:** job seekers rarely get realistic, personalised interview practice with immediate, structured feedback. Mock interviews with people are costly, hard to schedule and inconsistent.
- **Objective:** let anyone run a full technical and behavioural mock interview at any time. Questions come from their own resume or target role, answers can be spoken, and they get an actionable report.
- **Target users:** students and software developers preparing for technical interviews.
- **One-line pitch for the opening slide:** "Upload your resume, talk to an AI interviewer, get scored feedback in minutes."
- **Delivery:** a web app (Next.js, deployed on Vercel) plus a Windows desktop build (Electron) that loads the deployed site.

Screenshots for slides are in the repo's `ss/` folder (dashboard, login and 9 others).

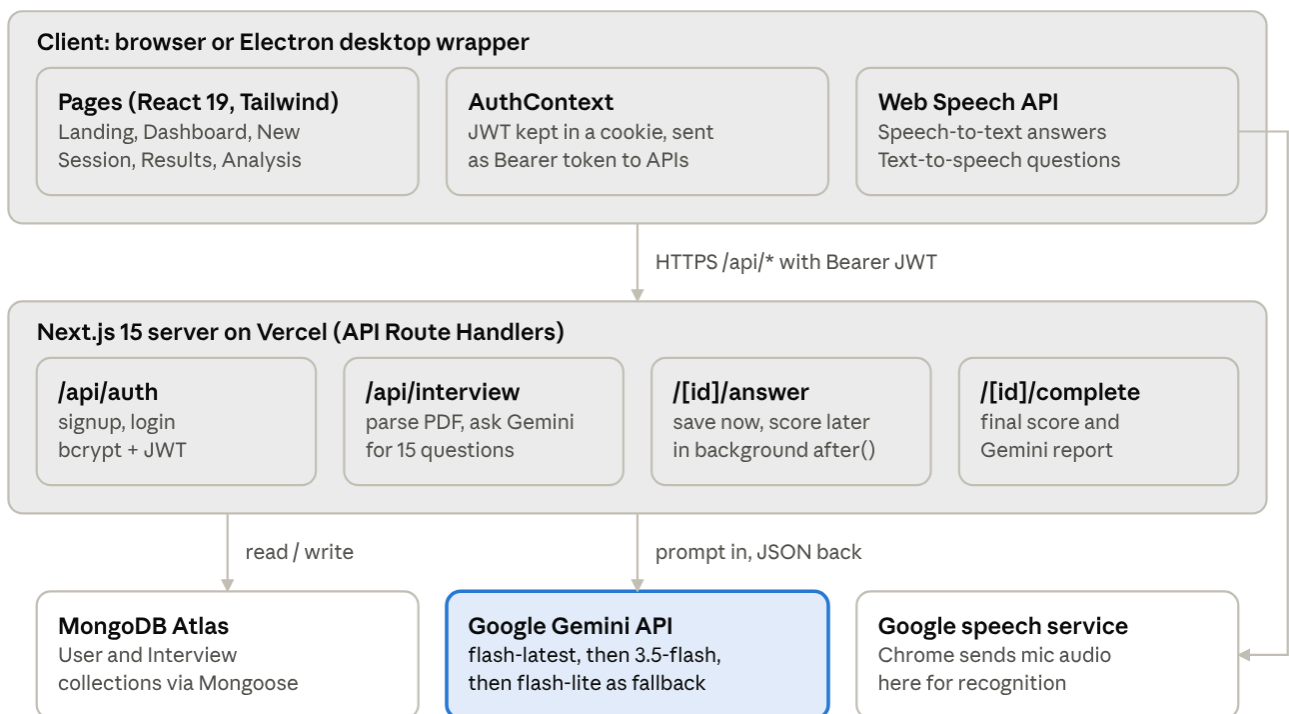
Tech stack and architecture

Interwise is one Next.js 15 app: React pages and the back-end API live in the same codebase and deploy together to Vercel. There is no separate Express server, even though the README says so.

Layer	Technology
Front end	Next.js 15 (App Router), React 19, TypeScript 5, Tailwind CSS v4, lucide-react icons
Back end	Next.js Route Handlers (<code>app/api/**/route.ts</code>)
Database	MongoDB (Atlas) with Mongoose 8
AI	Google Gemini via <code>@google/generative-ai</code> , JSON output mode

Layer	Technology
Speech	Browser Web Speech API (speech-to-text) and speechSynthesis (text-to-speech)
Resume parsing	pdf-parse
Auth	bcryptjs password hashing, jsonwebtoken (7-day JWT)
Hosting and desktop	Vercel; Electron 39 portable Windows .exe
Extras	Botpress web chat widget

One Next.js app talks to MongoDB and Gemini; speech runs in the browser



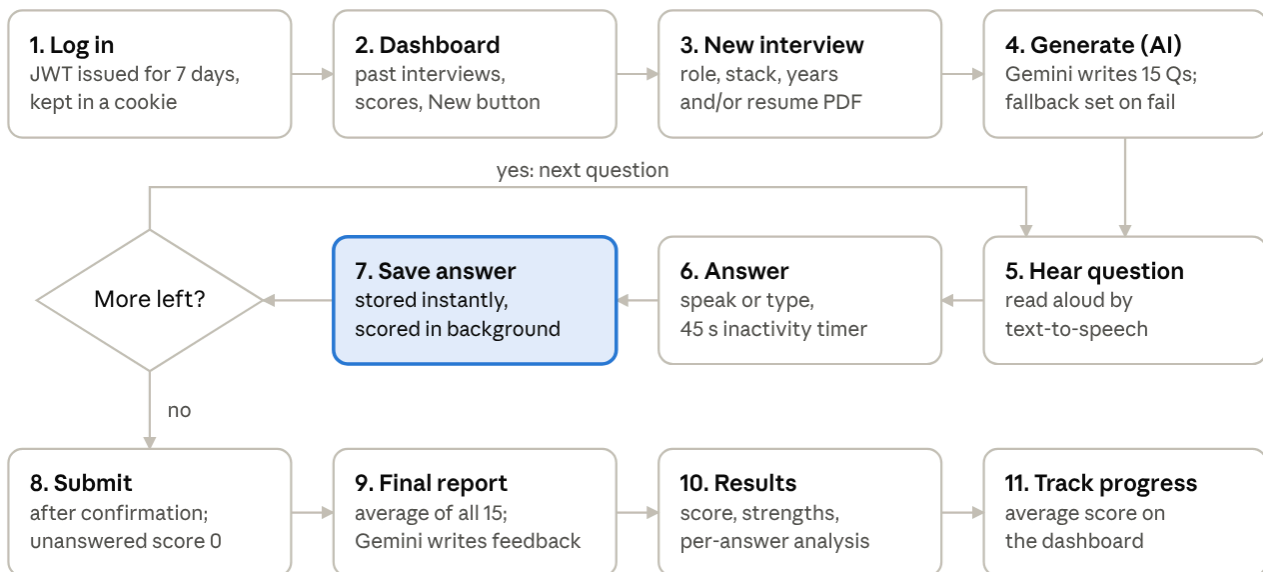
System architecture · 3 layers

The browser calls only Interwise's own API. The Gemini key and database stay on the server, while speech recognition goes directly from Chrome to Google.

End-to-end workflow

A user goes from login to a scored report in 11 steps. Steps 5–7 repeat for each of the 15 questions.

Each answer is saved at once and scored in the background



User workflow · 11 steps, 1 loop

What happens behind each step, useful when explaining the diagram:

Step	Page	API and logic
1. Log in / sign up	<code>/login</code> , <code>/signup</code>	<code>POST /api/auth/login</code> or <code>signup</code> : bcrypt check, JWT returned, saved in a cookie by <code>AuthContext</code>
2. Dashboard	<code>/dashboard</code>	<code>GET /api/interview/user</code> : stat cards and score rings (green 70+, amber 50+, red below)
3. New interview	<code>/interview/new</code>	Form posts role, stack, experience and resume (PDF up to 5 MB, or pasted text)
4. Generate	(loading)	<code>POST /api/interview</code> : extract PDF text, call Gemini, save Interview as in-progress
5-6. Hear and answer	<code>/interview/[id]</code>	Text-to-speech reads the question; the mic auto-starts; the transcript is editable
7. Save	same page	<code>POST /api/interview/[id]/answer</code> : saved now; scored by Gemini in <code>after()</code>

Step	Page	API and logic
8–9. Submit and report	confirm dialog	POST <code>/api/interview/[id]/complete</code> : blanks score 0, missing scores filled, final average, Gemini report
10. Results	<code>/interview/[id]/results</code> , <code>/analysis</code>	Overall score label, strengths, areas to improve, per-question feedback; print and share

A user who leaves mid-interview can click Continue on the dashboard. The session resumes at the first unanswered question.

Key features and how they work

The engineering story worth stressing is resilience. AI analysis runs in the background, Gemini falls back across models, and voice input recovers from browser glitches.

Feature	How it works	Where in code
Resume-based question generation	PDF text extracted with pdf-parse (5 MB max) or pasted text. Prompt asks Gemini for exactly 15 questions (10 technical, 5 soft-skill), harder as they go, returned as JSON.	<code>app/api/interview/route.ts</code> , <code>lib/gemini.ts</code> , <code>lib/pdfExtractor.ts</code>
Three context modes	Resume only, role + stack only, or both. Each mode has its own mix of soft-skill, technical, behavioural and project questions.	<code>lib/gemini.ts</code>

Feature	How it works	Where in code
Model fallback chain	Tries <code>GEMINI_MODEL</code> , then gemini-3.5-flash, then gemini-flash-lite-latest. 40 s per call, 120 s total. Retries 429/500 once; skips a failed model for 5 min.	<code>generateWithFallback</code> in <code>lib/gemini.ts</code>
Offline fallback questions	If every model fails, a built-in set (frontend, backend, fullstack, default) is used and an amber banner is shown.	<code>getFallBackQuestions</code>
Voice question and answer	Questions read aloud with the browser's <code>speechSynthesis</code> . Answers dictated with the Web Speech API (en-IN, en-US, en-GB) into an editable text box. Mic can start automatically after the question is read.	<code>components/interviewSession/InterviewSession.tsx</code>
45-second inactivity timer	Resets on typing or speech. Warning colours at 15 s and 5 s. On timeout it saves the draft and moves on.	<code>InterviewSession.tsx</code>

Feature	How it works	Where in code
Background answer scoring	Answer saved instantly; Gemini scores it after the response using Next 15 <code>after()</code> . Weights: technical accuracy 40%, communication 30%, problem-solving 20%, best practices 10%.	<code>app/api/interview/[id]/answer/route.ts</code>
Stale-result guard	A score is stored only if the saved answer still matches the text that was scored, so an edited answer is never given an old score.	same route, <code>updateOne</code> filter
Scoring rules	In progress: average of scored answers. Completed: average over all 15, unanswered = 0.	<code>lib/interviewScore.ts</code>
Final report	Gemini writes overall feedback, strengths, areas to improve and next steps from all answers and scores.	<code>app/api/interview/[id]/complete/route.ts</code>

Feature	How it works	Where in code
Dashboard and results	Stat cards (total, completed, in progress, average score), colour-coded score rings, results tabs, per-question analysis page, print and share.	<code>components/interview/InterviewList.tsx</code> , <code>app/interview/[id]/results</code> , <code>/analysis</code>
Authentication	Email and password, bcrypt hashing, 7-day JWT sent as a Bearer token. Every API route checks the token and ownership.	<code>app/api/auth/*</code> , <code>lib/auth.ts</code> , <code>context/AuthContext.tsx</code>

Data model (MongoDB): `User` holds name, email and password hash. `Interview` holds the user, role, stack, experience, resume text, a status (in-progress or completed), the overall score and the feedback. It also holds the 15 questions, each with its answer and analysis.

Seminar flow and demo script

Plan for about 15 slides in 15–20 minutes: 10 minutes of slides, 5–7 minutes of live demo, then questions.

1. **Title:** Interwise, an AI-powered mock interview platform, with team names and guide.
2. **Problem statement:** interview anxiety, no personalised practice, slow or no feedback.
3. **Objectives:** personalised questions, voice-based answering, instant scored feedback, progress tracking.
4. **Existing systems vs Interwise:** generic question banks and human mock interviews compared with resume-aware AI questions and per-answer scoring.
5. **Tech stack:** Next.js 15, React 19, TypeScript, Tailwind, MongoDB, Google Gemini, Web Speech API, JWT, Vercel, Electron.
6. **System architecture:** the diagram below.
7. **Workflow:** the flow diagram below.

8. **Question generation:** the three context modes, the 15-question JSON prompt and the fallback chain.
9. **Voice interview:** text-to-speech, speech-to-text, auto-start mic and the 45 s timer.
10. **Evaluation and scoring:** the 40/30/20/10 rubric, background scoring with `after()`, and how the final score is calculated.
11. **Database design:** the User and Interview schemas.
12. **Results and screenshots:** dashboard, session, results and analysis pages from `SS/`.
13. **Live demo** (script below).
14. **Challenges, limitations and future scope.**
15. **Conclusion and Q&A.**

Live demo script (5–7 minutes)

1. Open the landing page, then log in with a prepared demo account.
2. Show the dashboard: stat cards and a past completed interview with its score ring.
3. Click New Interview. Enter a role such as "Frontend Developer", the stack "React, TypeScript" and 1 year of experience, then upload a sample resume PDF.
4. While the questions generate (up to about a minute), explain the Gemini prompt and fallback chain.
5. Answer 2 questions: one by voice (show the live transcript) and one by typing. Point out the timer.
6. Submit early. Explain that unanswered questions score 0.
7. Show the results page (score, strengths, areas to improve), then the per-question analysis page.

Demo checklist

- ☐ Use Google Chrome; Brave and Electron can block speech recognition
- ☐ Allow microphone access beforehand and test in the seminar room
- ☐ Use a stable network with no VPN; the speech service and Gemini both need internet
- ☐ Check the Gemini API key quota the day before
- ☐ Pre-create a completed interview so results can be shown if live generation is slow
- ☐ Keep screenshots or a screen recording as a backup if Wi-Fi fails
- ☐ Keep a sample resume PDF under 5 MB ready on the desktop

Challenges, limitations and future scope

Challenges solved (good talking points)

- **Slow AI scoring:** submitting an answer used to block for 10–45 seconds. Scoring now runs in the background after the response is sent.
- **Gemini rate limits and outages:** solved with a multi-model fallback chain, timeouts, model health tracking and built-in questions.
- **Unreliable browser speech recognition:** silent auto-restart, network retries, and a 1.5 s flush so the last words before Submit are not lost.
- **Race conditions:** a score is discarded if the answer changed while it was being scored.
- **MongoDB Atlas on Windows:** SRV DNS lookups failed; fixed by forcing public DNS servers.

Current limitations (be honest if asked)

- Voice input works only in Chromium browsers with internet access. Questions are read aloud in en-US only.
- AI scoring is subjective and depends on the model; there is no human-validated benchmark.
- The fallback question sets have 6 questions, not 15.
- Learning resources are generated but not saved (missing schema field), so that section does not show.
- The analysis page has slots for per-dimension scores and a sample ideal answer that the AI does not return yet.
- Settings and Google/Facebook login are placeholders.
- Security hardening is pending before production: the debug routes are open, the JWT sits in a non-HttpOnly cookie, and there is no rate limiting.
- There are no automated tests, and builds skip type and lint errors.

Future scope

- Video interview with facial expression and eye-contact analysis (MediaPipe and TensorFlow.js are already in the dependencies).
- Per-dimension scores (technical, communication), ideal sample answers and learning resources saved per interview.
- Server-side speech-to-text (for example Whisper) for all browsers, and multilingual support.
- Company-specific and coding-round interviews with a live code editor.

- Progress analytics over time, and Google OAuth login.
- HttpOnly cookies, rate limiting, input validation with zod, and automated tests.

Likely questions and answers

Question	Suggested answer
Why Gemini and not ChatGPT?	It has a generous free tier, fast Flash models and native JSON output (<code>responseMimeType: application/json</code>), which makes parsing reliable.
How do you make sure the AI returns valid output?	We request JSON only, strip code fences, parse it, and check the fields. If generation fails, we retry other models and finally fall back to built-in questions.
How is an answer scored?	Gemini scores 1–100 with a fixed rubric: technical accuracy 40%, communication 30%, problem-solving 20%, best practices 10%. It also gives technical feedback, communication feedback and suggestions.
How is the final score calculated?	The average over all questions, with unanswered ones counted as 0.
Is the AI scoring reliable?	It is consistent enough for practice feedback but not a hiring decision. A fixed rubric and temperature 0.7 keep it stable. Validating it against human scorers is future work.
How does voice input work?	The browser's Web Speech API converts speech to text live. The transcript is editable before saving. Questions are read aloud with speechSynthesis.
Why does voice fail in some browsers?	Chrome sends audio to Google's speech service. Brave, Electron and some VPNs block it, so we show a clear error and text input still works.
How is user data secured?	Passwords are hashed with bcrypt. Sessions use 7-day JWTs. Every API call checks the token and that the interview belongs to the user.

Question	Suggested answer
Why Next.js for front end and back end?	One codebase and one deployment on Vercel. API routes run server-side, so the Gemini key never reaches the browser.
Why MongoDB?	An interview is naturally one document with nested questions, answers and analysis. A flexible schema suits AI output that changes shape.
What happens if the user closes the tab mid-interview?	Each answer is saved as it is submitted. The dashboard shows Continue, which resumes at the first unanswered question.
How long does question generation take?	Usually a few seconds and at most about a minute. There is a 40 s timeout per model and 120 s overall.
What is the Electron app?	A Windows desktop wrapper around the deployed website, packaged as a portable .exe.
How is it different from existing platforms?	Questions come from your own resume, you can answer by voice, and you get scored feedback per answer right away, for free.